

VIRGINIA TECH UNIVERSITY
AOE



Adaptive Control

Final Report

Adaptive Control of a Ground Vehicle With Chrono Simulation

Instructor: Dr. L'Afflitto

Student Name: Barr Shlagman

BLACKSBURG, MAY 2026



Contents

1	Introduction	5
2	Scope	5
3	The Bicycle Model	5
3.1	Model Derivation	5
3.2	The Control Law	7
3.3	Notes on Chrono Integration	8
4	Testing	9
4.1	Straight Line	9
4.2	Circular Path (Figure Eight)	13
4.3	Notes on Tuning	16
5	Conclusion	17

Figures

3.1	Bike model [1]	6
4.1	XY path of the HMMWV	10
4.2	x (v_1) and y (v_2) velocities at the rear-axle of the HMMWV (m/s)	10
4.3	Inner and outer loop errors for the tracked states	11
4.4	Plot of the steering command. The zoomed in portion shows the high frequency oscillations although they display better in the dynamic python plots.	11
4.5	Yaw-rate of the HMMWV during the maneuver. The effects of the steering oscillations can be seen clearly when the vehicle straightens out.	12
4.6	Inner and outer loop adaptive gain norms	12
4.7	The HMMWV attempts to follow a circular trajectory with a direction change but is unable to track the user trajectory. It should be noted that r_{cmd} tracks the position well and the vehicle follows the command. Due to it's inability to get up to the required speed fast enough, the HMMWV steers to meet the command late and does not successfully follow the direction change.	13



4.8	The delayed velocity response can be clearly seen here. With a direct drive motor at the wheels, an instantaneous response could help the vehicle track the velocity better. Although sharp transitions would still pose a challenge.	14
4.9	The error of the inner and outer loops oscillates as the controller attempts to drive it down but it never converges to 0.	14
4.10	The steering input of the HMMWV to track the circular path.	15
4.11	The resulting yaw-rate of the HMMWV.	15
4.12	Inner and outer loop gain norms. The inner loop norm continues to grow throughout the maneuver.	16

Listings

4.1	$\dot{r}_{cmd}$ and $\ddot{r}_{cmd}$ tuning example in python	16
-----	---	----



1 Introduction

Autonomous navigation of ground vehicles is a growing area of research. One of the fundamental challenges is controlling the vehicle. Not only are most ground vehicles under-actuated, the governing dynamics can be highly nonlinear at the edge of the operational envelope with time-varying or unknown parameters. A hierarchical control architecture is typically employed where PID's or similar variants govern the lower level dynamics of the actuators and a supervisory level controller, such as Model Predictive Control, governs the system level dynamics, making requests from the actuators to meet the desired trajectory while respecting constraints based on the physical capabilities of the system. Adaptive control has not been used as frequently to control vehicles for trajectory tracking. It poses a few benefits such as adapting to parameter uncertainty and handling nonlinear behavior, all aspects that pertain to ground vehicles, and will be studied through numerical simulation of the HMMWV in Pychrono.

2 Scope

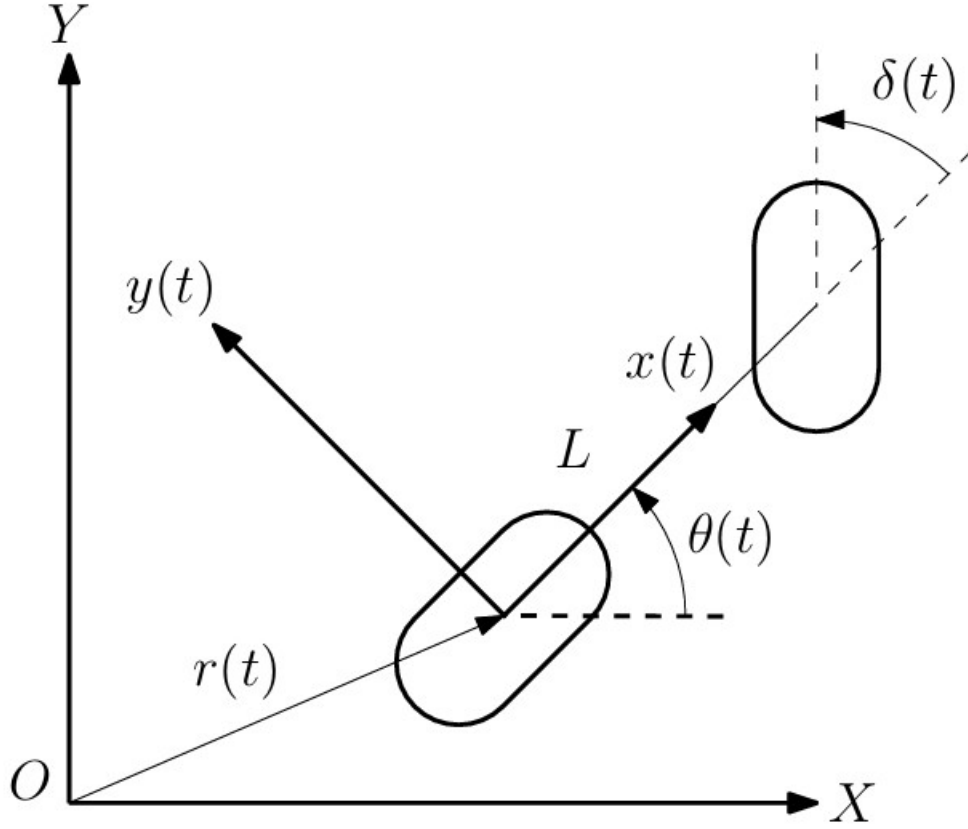
The scope of this report will develop an analysis of the following,

- A brief re-derivation with application of a robust adaptive control law for a compatible model of a ground vehicle.
- Testing the controller on different trajectories with the available HMMWV vehicle class in Pychrono.
- Review of the performance of the controller.

3 The Bicycle Model

3.1 Model Derivation

The bicycle model of a ground vehicle lumps both front and rear wheels together as a single track. In [1], only the front axle is capable of a steering input (δ) and a tractive force input (u_{force}) to the rear wheel (effectively a rear wheel drive vehicle). The local coordinate frame of the bicycle is defined at the rear wheel ($r(\cdot)$) with v_1 and v_2 as the respective velocity components.



3.1: Bike model [1]

The dynamics can be written as,

$$\dot{r}(t) = \begin{bmatrix} \cos \theta(t) \\ \sin \theta(t) \end{bmatrix} v_1(t), \quad r(t_0) = r_0, \quad t \geq t_0, \quad (3.1)$$

$$\dot{\theta}(t) = L^{-1} v_1(t) \tan \delta(t), \quad \theta(t_0) = \theta_0, \quad (3.2)$$

$$\dot{v}_1(t) = m^{-1} [u_{force}(t) - b_1 v_1(t) - b_2 v_1(t) |v_1(t)|], \quad v_1(t_0) = v_0, \quad (3.3)$$

The mass ($m > 0$), wheelbase ($L > 0$) and coefficients of friction and drag, respectively ($b_1 > 0, b_2 > 0$) are all unknown.

Given the objective of tracking a user defined, twice differential trajectory,

$$\lim_{t \rightarrow \infty} \|r(t) - r_{user}(t)\| = 0, \quad (3.4)$$

$$\lim_{t \rightarrow \infty} \|\dot{r}(t) - \dot{r}_{user}(t)\| = 0. \quad (3.5)$$

a control law must be designed to generate steering and force inputs to meet the objective.



3.2 The Control Law

Due to the underactuated nature of the model (3 degrees of freedom with two control inputs), an outer loop is designed to both capture the dynamics in MRAC form and overcome the underactuated nature of the vehicle. A commanded input (r_{cmd}) to track the user defined position with the reference model can be designed with a PI controller as follows,

$$\dot{r}_{cmd} = -K_{P,tran,kin}(r(t) - r_{user}(t)) - K_{I,tran,kin} \int_0^t (r(\tau) - r_{user}(\tau))d\tau + \dot{r}_{user} \quad (3.6)$$

Then introducing $\tilde{v}(t) = [\tilde{v}_1, \tilde{v}_2]^T$ represented in the orthonormal body reference frame, $\mathbb{J}$,

$$\dot{\tilde{v}}(t) = m^{-1} \left(\tilde{u}_{force}(t) - m\dot{\theta}(t)[- \tilde{v}_2, \tilde{v}_1]^T + F_{friction}(t) \right), \quad (3.7)$$

$$F_{friction} = m\dot{\theta}(t)[- \tilde{v}_2, \tilde{v}_1]^T + F_{friction,tangential}(t), \quad (3.8)$$

which reduces to,

$$\dot{\tilde{v}}(t) = m^{-1} (\tilde{u}_{force}(t) + F_{friction,tangential}(t)) \quad (3.9)$$

Given the transformation to global coordinates hold, then 3.9 is in MRAC form.

To compute $\tilde{u}_{force}$ the reference velocity can be computed using a reference model as follows,

$$\dot{v}_{ref}(t) = A_{ref,tran}v_{ref}(t) + \rho_{tran}(t), \quad v_{ref}(t_0) = \dot{r}_{cmd}(t_0), \quad t \geq t_0, \quad (3.10)$$

$$\begin{aligned} \rho_{tran}(t) \triangleq & \ddot{r}_{cmd}(t) - A_{ref,tran}\dot{r}_{cmd}(t) - K_{P,tran,dyn} \int_{t_0}^t (v_{ref}(\tau) - \dot{r}_{cmd}(\tau))d\tau \\ & - K_{I,tran,dyn} \int_{t_0}^t \int_{t_0}^{\tau} (v_{ref}(\sigma) - \dot{r}_{cmd}(\sigma))d\sigma d\tau, \end{aligned} \quad (3.11)$$

where $\ddot{r}_{cmd}$ is given by differentiating $\dot{r}_{cmd}$,

$$\ddot{r}_{cmd} = -K_{P,tran,kin}(\dot{r}(t) - \dot{r}_{user}(t)) - K_{I,tran,kin}(r(t) - r_{user}(t)) + \ddot{r}_{user}(t) \quad (3.12)$$

Once $\tilde{u}_{force}$ is calculated,

$$u_{force}(t) = \|\tilde{u}_{force}(t)\| \quad (3.13)$$

$$\theta_{cmd}(t) \triangleq \text{atan2}(\tilde{u}_{force,2}(t), \tilde{u}_{force,1}(t)) \quad (3.14)$$

The Inner loop MRAC laws will command the steering to enable the vehicle to follow the



force commanded at the rear wheel so,

$$\lim_{t \rightarrow \infty} |\theta(t) - \theta_{cmd}(t)| = 0 \quad (3.15)$$

rewriting 3.2 as,

$$\dot{\theta}(t) = L^{-1}u_{moment}(t), \quad \theta(t_0) = \theta_0 \quad (3.16)$$

$$u_{moment}(t) \triangleq v_1(t) \tan \delta(t) \quad (3.17)$$

u_{moment} must be calculated using MRAC to guarantee tracking of the reference rotational model given the unknown value of L . The reference model is a single equation with scalar values for $A_{ref,rot}$ and ρ_{rot} defined as follows,

$$\dot{\theta}_{ref}(t) = a_{ref,rot}\theta_{cmd}(t) + \rho_{rot}(t), \quad \theta_{ref}(t_0) \quad (3.18)$$

$$\rho_{rot}(t) \triangleq -a_{ref,rot}\theta_{cmd} - K_{I,rot,kin} \int_{t_0}^t (\theta_{ref}(\tau) - \theta_{cmd}(\tau))d\tau + \dot{\theta}_{cmd}(t) \quad (3.19)$$

Finally, the steering input can be calculated,

$$\delta(t) = \text{atan2}(u_{moment}(t), v_1(t)), \quad t \geq t_0 \quad (3.20)$$

Since $\dot{\theta}_{cmd}$ cannot be calculated analytically, [1] proposes a second order low pass filter to differentiate θ_{cmd} with user defined natural frequency (ω) and damping (ζ) parameters. θ_{cmd} is also unwrapped during runtime to prevent oscillations between $-\pi$ and π .

For robustness, the simulation was tested with the continuous convex projection operator applied to the control law.

3.3 Notes on Chrono Integration

The available Chrono vehicle class for simulation provides multiple vehicles and configurations to select from. For simulation purposes, the HMMWV was chosen. A flat terrain was chosen to test on since the current model assumptions only hold for flat terrain. One core difference that must be addressed is the control input configuration in Chrono. While the vehicle can be configured with rear wheel drive, it is not straight-forward to input a force command at the rear. It is possible to multiply the force command by the radius of the tire to get a torque input request, however, that request must be mapped to a throttle input which is subject to many nonlinearities and delays from the engine and driveline.



An ideal configuration to test the controller would be a rear wheel drive, RC-scale vehicle, that has a single small motor driving both wheels with minimal delays. To overcome some of the challenges in mapping the control command, a PI feedback controller was used to command the throttle to track the desired longitudinal velocity at the rear-axle center. A central issue of this hierarchical controller structure is that $\theta_{cmd}(t)$ which directly impacts the steering control command, is dependent on u_{force} . Therefore, since u_{force} is effectively bypassed, there is a mismatch in the controller that may erode the stability guarantees of the MRAC structure.

4 Testing

4.1 Straight Line

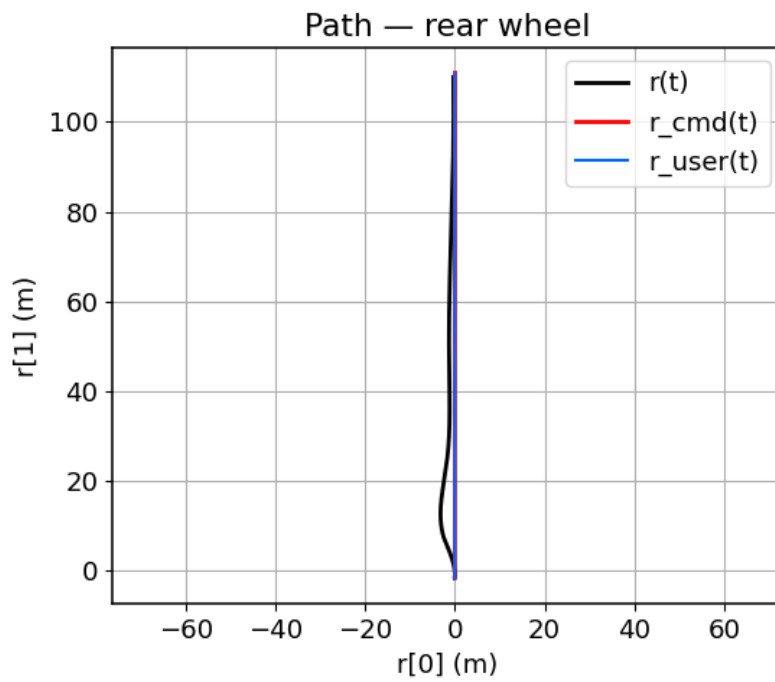
The first trajectory to be tested is a straight line in the XY plane along the positive Y direction. The velocity ramps from 0 to a user defined final velocity and then holds the velocity constant for the duration of the simulation. The simple kinematic relationships needed to obtain this trajectory are as follows when $\dot{r}_{user} < v_f$ for a user defined v_f over a user defined velocity ramp period, T_{ramp} ,

$$\ddot{r}_{user}(t) = \begin{cases} v_f/T_{ramp} & \dot{r}_{user} < v_f, \\ 0 & \text{otherwise,} \end{cases} \quad (4.1)$$

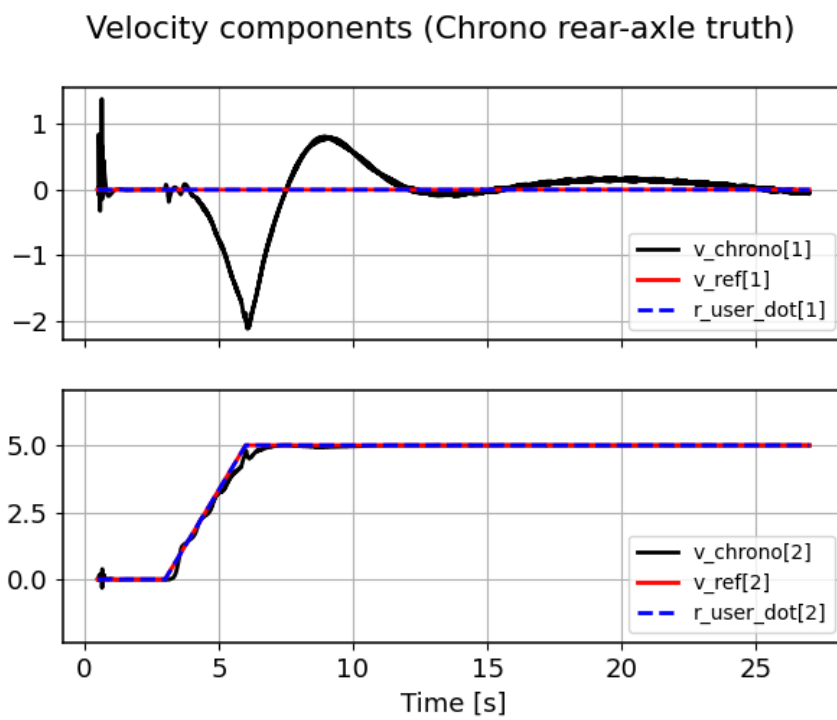
$$\dot{r}_{user}(t) = \begin{cases} \ddot{r}_{user} t & \dot{r}_{user} < v_f, \\ v_f & \text{otherwise,} \end{cases} \quad (4.2)$$

$$r_{user}(t) = \begin{cases} \frac{1}{2}\ddot{r}_{user}t^2 & \dot{r}_{user} < v_f, \\ \underbrace{0.5v_fT_{ramp}}_{\text{Integral during ramp}} + v_f(t - T_{ramp}) & \text{otherwise.} \end{cases} \quad (4.3)$$

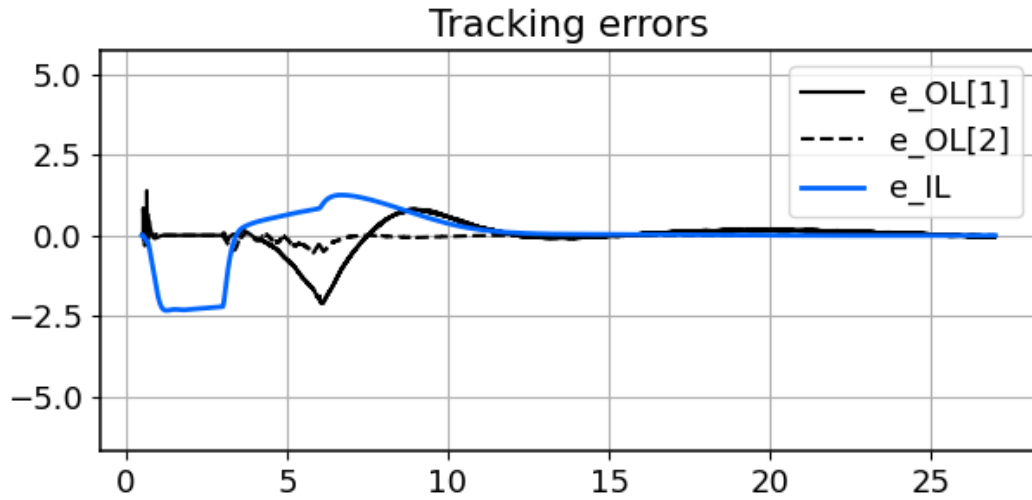
The control inputs were able to track the trajectory allowing the plant to converge with the reference model response and follow the user defined trajectory.



4.1: XY path of the HMMWV

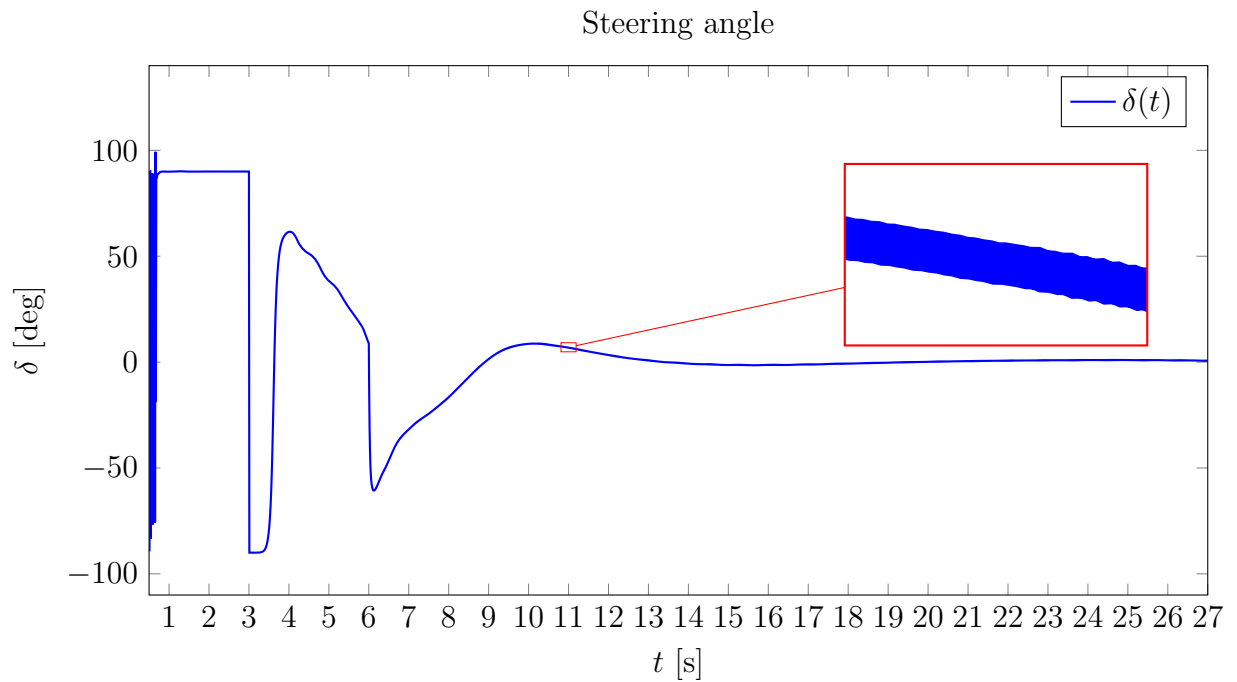


4.2: x (v_1) and y (v_2) velocities at the rear-axle of the HMMWV (m/s)

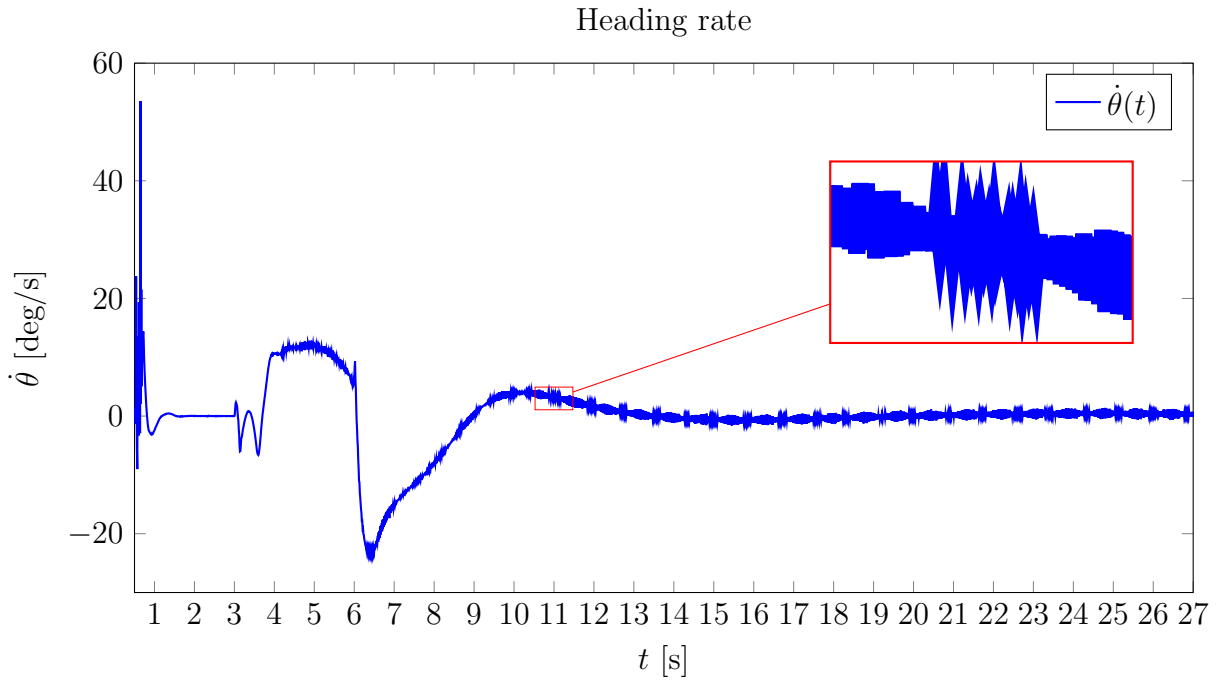


4.3: Inner and outer loop errors for the tracked states

The steering input had an aggressive initial input that was clipped by chrono due to steering limits of the HMMWV. The adaptive gains commanded the steering response with high frequency oscillations to attempt to track the reference trajectory closely. The yaw-rate response was affected pretty heavily by the steering input oscillations.

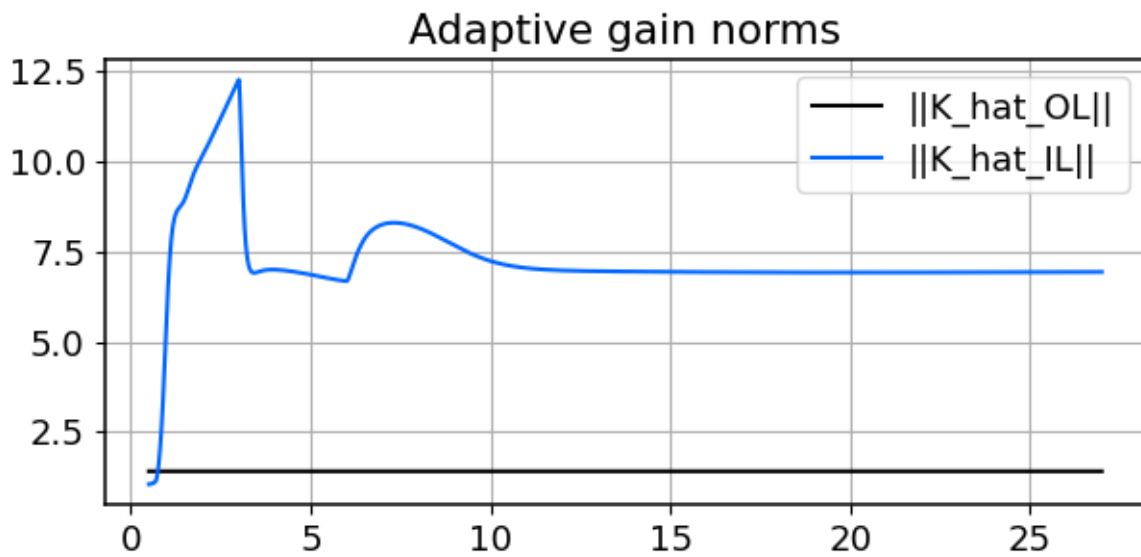


4.4: Plot of the steering command. The zoomed in portion shows the high frequency oscillations although they display better in the dynamic python plots.



4.5: Yaw-rate of the HMMWV during the maneuver. The effects of the steering oscillations can be seen clearly when the vehicle straightens out.

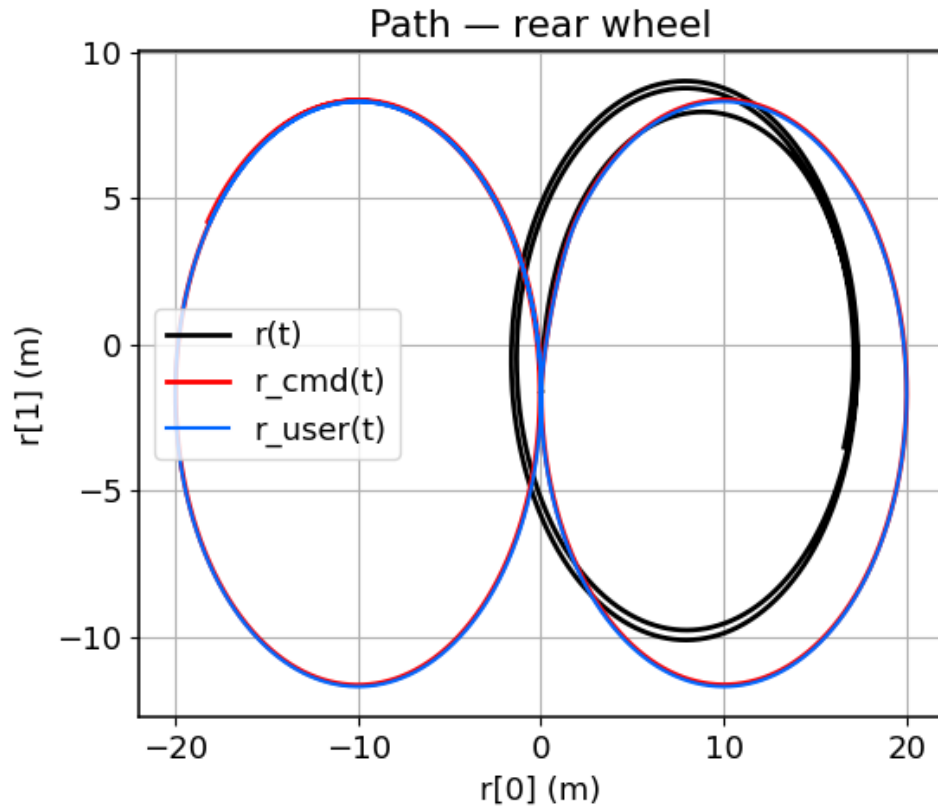
The adaptive gain norms for the inner and outer loop remained reasonable throughout the maneuver, showing a successful implementation of the continuous convex projection operator to avoid parameter drift.



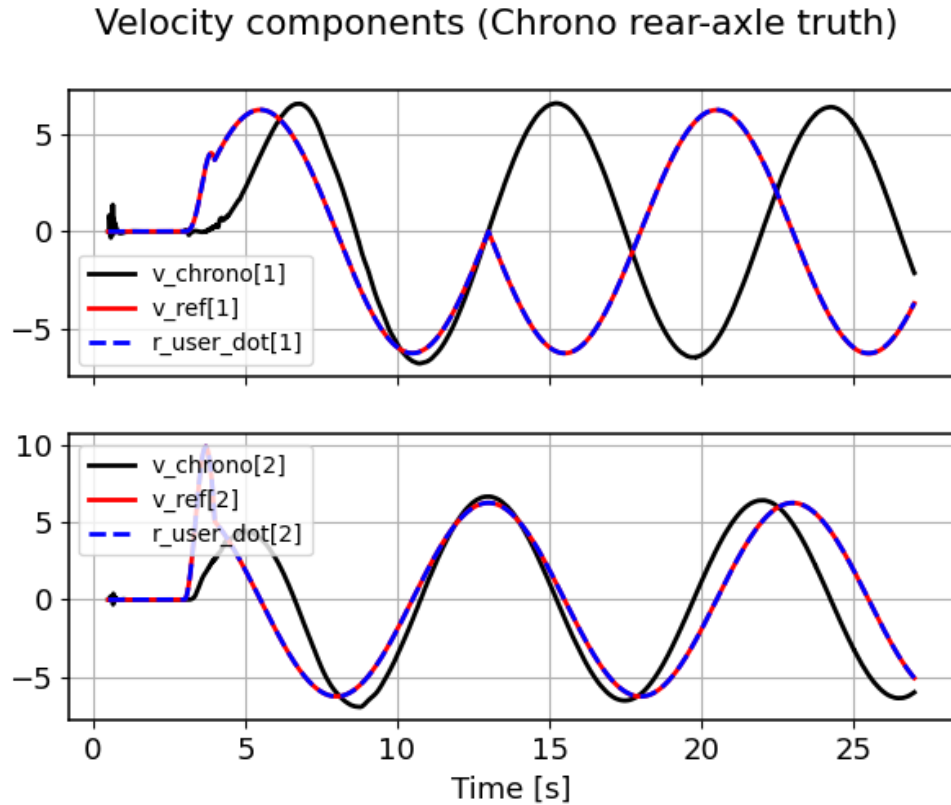
4.6: Inner and outer loop adaptive gain norms

4.2 Circular Path (Figure Eight)

In an attempt to get the HMMWV to track the circular trajectory, the difficulties of implementing unrealistic trajectories was highlighted. A large issue of this trajectory is that the velocity is non-zero initially requiring a large force input (mapped to throttle with PID controller). To mitigate the large input, the trajectory was modified to ramp the velocities more gradually. This still posed limitations as the delays in the HMMWV response increased the tracking errors, making it difficult for the controller to converge and track the trajectory.

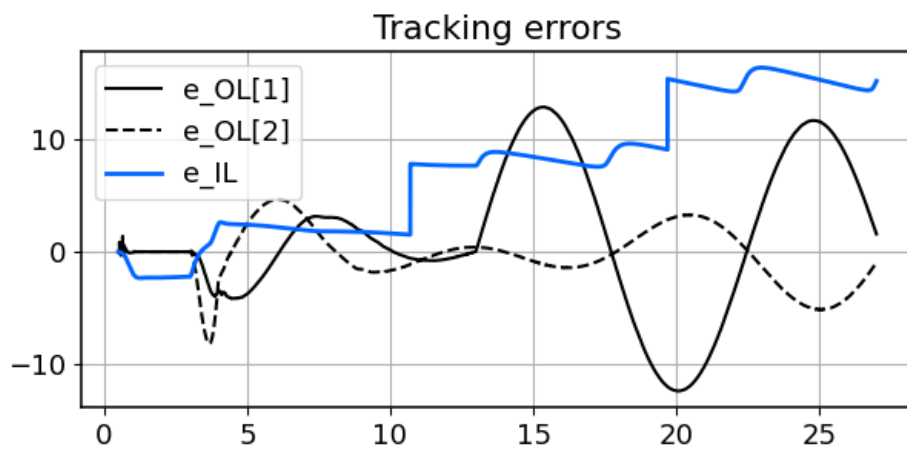


4.7: The HMMWV attempts to follow a circular trajectory with a direction change but is unable to track the user trajectory. It should be noted that r_{cmd} tracks the position well and the vehicle follows the command. Due to its inability to get up to the required speed fast enough, the HMMWV steers to meet the command late and does not successfully follow the direction change.



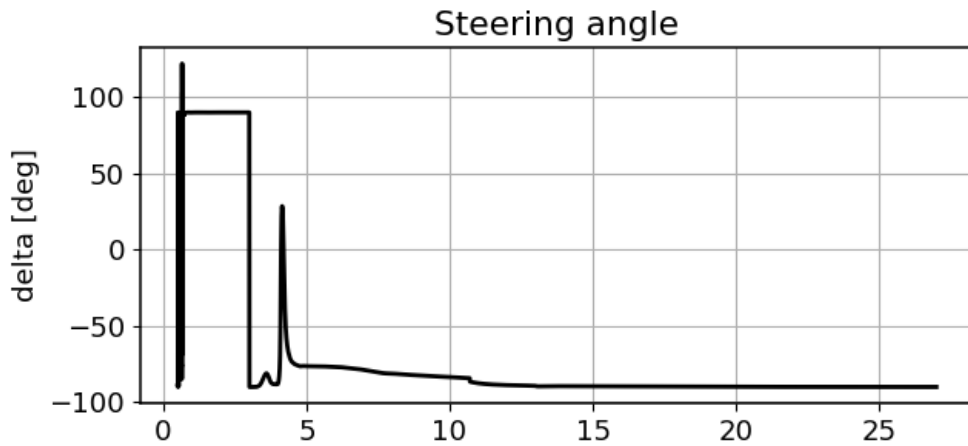
4.8: The delayed velocity response can be clearly seen here. With a direct drive motor at the wheels, an instantaneous response could help the vehicle track the velocity better. Although sharp transitions would still pose a challenge.

While not exceeding an overly large value, there is constant error throughout the maneuver.

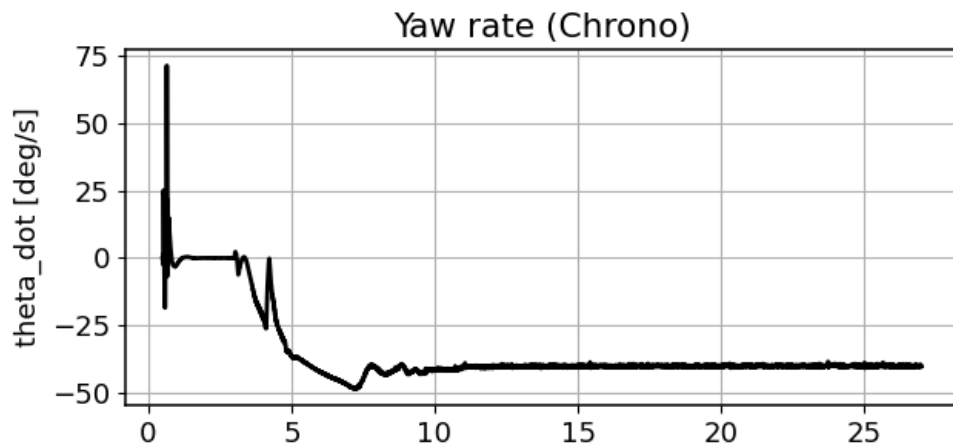


4.9: The error of the inner and outer loops oscillates as the controller attempts to drive it down but it never converges to 0.

The steering input drives a yaw rate response that tracks a semblance of the path but does not directly follow it.

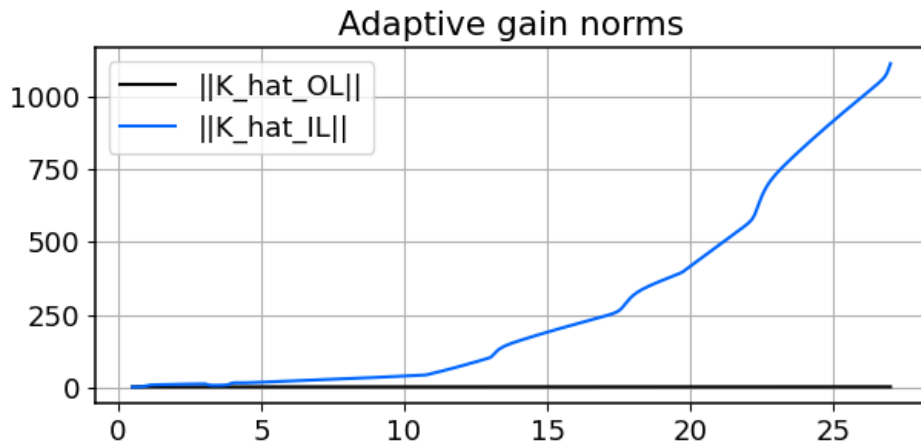


4.10: The steering input of the HMMWV to track the circular path.



4.11: The resulting yaw-rate of the HMMWV.

The norm of the adaptive gains shows that the controller struggles to track this trajectory. The gains do not converge easily and seem to continue to grow. The norm is bounded by the projection operator, but the inner loop norm grows significantly.



4.12: Inner and outer loop gain norms. The inner loop norm continues to grow throughout the maneuver.

4.3 Notes on Tuning

For the commanded signals to converge to the user defined trajectory for the reference model, the user defined gains were tuned to small values.

Listing 4.1: $\dot{r}_{cmd}$ and $\ddot{r}_{cmd}$ tuning example in python

```
1
2 # 3. MRAC parameters
3 -----
4
5 .0;    Lbar = 3.4
6 m_p   = 1.0;    mbar = 4500
7
8 parameters = {
9     'L': L, 'Lbar': Lbar, 'm': m_p, 'mbar': mbar,
10    'M': np.eye(1), 'emax': 7.0,
11 }
12
13 K_P_tran_kin = 0.0001 * np.eye(2)
14 K_I_tran_kin = 0.0001 * np.eye(2)
15 A_ref_tran   = -.05 * np.eye(2)
16 K_P_tran_dyn = 0.1 * np.eye(2)
17 K_I_tran_dyn = 0.10 * np.eye(2)
```



The adaptive gains also required relatively small values. Issues with the RK4 integrator became prevalent with large values, even causing the car to glitch and phase through the terrain. With the fixed step ODE solver, it was also imperative to set the natural frequency in the low-pass filter to a low and reasonable value.

5 Conclusion

Applying an MRAC controller structure successfully to ground vehicles such as the HMMWV presents many benefits. The nonlinear dynamics paired with a multitude of unknown or difficult to measure parameters make this system a good candidate. Yet there can be several challenges that need to be considered when implementing this controller architecture. Namely, the actuator inputs must be mapped correctly given the model. Furthermore, with complex engine and driveline dynamics present in most modern vehicles, it is important to understand the effects of the unmodeled dynamics such as driveline or engine losses and delays in the system.

Using the HMMWV as a testing platform in chrono, it was shown that MRAC can be implemented successfully with a reasonable trajectory and tuning. The model that was used to develop the MRAC controller holds only under the assumption that there is no slip and the vehicle is driving on a flat plane. Future work can be done to develop a model that accounts for wheel slip to enable more aggressive maneuvers. The model can also try to account for a tractive force mapping for throttle input with a structure that holds for MRAC.



References

- [1] Andrea L’Afflitto. *Notes on Robust Adaptive Control and Observation*. Springer, February 2026.